

Consider the following schedule:

- Begin_checkpoint
- End_checkpoint
- T10: writes DEF to Page P500 from 2nd byte
- T20: writes KLM to Page P600 from 1st byte
- T20: writes QRS to Page P500 from 1st byte
- T20: commit
- T10: writes WXY to Page P505 from 2nd byte
- Crash, restart

Recover the database using the ARIES Algorithm.

Consider LSN 0 from Begin_checkpoint. Increment next LSN by 1.

Assume each page is of 4 bytes.

Consider contents of pages P500, P505, and P600 as GABC, STUV and HIJK respectively, and pageLSN of all pages -1 before Begin_checkpoint.

Assume that before the crash only P600 is copied to the hard disk.

LOG TABLE before crash								
LSN	PREV LSN	Xn ID	STATUS	PAGE ID	LENGTH	OFFSET	BEFORE IMAGE	AFTER IMAGE
2	NULL	T10	UPDATE	P500	4	2 nd byte	GABC	GDEF
3	NULL	T20	UPDATE	P600	4	1 st byte	HIJK	KLMK
4	3	T20	UPDATE	P500	4	1 st byte	GDEF	QRSF
5	4	T20	COMMIT					
6	2	T10	UPDATE	P505	4	2 nd byte	STUV	SWXY
7	endchkpt							

LOG TABLE after Recovery								
LSN	PREV LSN	Xn ID	STATUS	PAGE ID	LENGTH	OFFSET	BEFORE IMAGE	AFTER IMAGE
2	NULL	T10	UPDATE	P500	4	2 nd byte	GABC	GDEF
3	NULL	T20	UPDATE	P600	4	1 st byte	HIJK	KLMK
4	3	T20	UPDATE	P500	4	1 st byte	GDEF	QRSF
5	4	T20	COMMIT					
6	2	T10	UPDATE	P505	4	2 nd byte	STUV	SWXY
7	endchkpt							
8		T10	Undoing an update	P505			SWXY	STUV

9	8	T10	CLR, undoNextLSN = 2					
10		T10	Undoing an update	P500			QRSF	GABC
11	10	T10	CLR, undoNextLSN = NULL					
12		T10	END					

DPT after the analysis/redo phase (ie after recovery)	
PAGE ID	REC LSN
P500	2
P505	6
P600	3

TT after the analysis phase (ie after recovery)	
Xn ID	LAST LSN
T10	6

(writing T20 in TT is incorrect as it is no more an active transaction)

Page contents and PAGELSN when page P500 and P600 are written to the disk.

PAGE ID	PAGELSN BEFORE SCHEDULE	PAGE CONTENTS AFTER CRASH	PAGELSN AFTER CRASH	PAGE CONTENTS AFTER REDO PHASE	PAGELSN AFTER RECOVERY/ REDO	PAGE CONTENTS AFTER UNDO PHASE
P500	-1	QRSF	4	QRSF	4	QRSF
P505	-1	STUV	-1	SWXY	6	STUV
P600	-1	KLMK	3	KLMK	3	KLMK

Page contents and PAGELSN when only page P600 is written to the disk.

PAGE ID	PAGELSN BEFORE SCHEDULE	PAGE CONTENTS AFTER CRASH	PAGELSN AFTER CRASH	PAGE CONTENTS AFTER REDO PHASE	PAGELSN AFTER RECOVERY/ REDO	PAGE CONTENTS AFTER UNDO PHASE
P500	-1	GABC	-1	GDEF then QRSF	2, 4	GABC
P505	-1	STUV	-1	SWXY	6	

						STUV
P600	-1	KLMK	3	KLMK	3	KLMK

In Redo phase: look at the DPT and start with the smallest recLSN = 2 (here) because this log record identifies the oldest update that may not have been written to the disk prior to the crash.

The page to be updated is P500. Its pageLSN = -1 which is less than recLSN=2. Thus the update is performed (bytes 21 through 23 are changed to DEF) and the pageLSN is set to the LSN of this update log record = 2.

No additional log record is written.

Redo then examines the second log record (=3 here). Again, the affected page, P600, is fetched and the pageLSN (3) is compared to the LSN of the update log record. In this case, because we assumed that P600 was written to disk before the crash, they are equal, and **the update does not have to be redone.**

Redo now examines the log with recLSN = 4. The affected page is P500. Its pageLSN (2) is compared with recLSN = 4, as pageLSN < recLSN, the update is performed. (bytes 20 through 22 are changed to QRS) and the pageLSN is set to the LSN of this update log record = 4. **No additional log record is written.**

Redo examines the next log record with recLSN = 5. The status is commit so proceeds.

Redo examines the next log record with recLSN = 6. The affected page is P505. Its pageLSN (-1) is compared with recLSN = 6, as pageLSN < recLSN, the update is performed. (bytes 21 through 23 are changed to WXY) and the pageLSN is set to the LSN of this update log record = 6. No additional log record is written.

In UNDO Phase: first construct the ToUndo set = the set of lastLSN values for all loser (active) transactions (those who did not commit) looking at the Transaction table.

ToUndo = { 6 }

Undo repeatedly chooses the largest (ie., most recent) LSN value in this set and processes it, until the set is empty.

It undoes all the action backwards.

At recLSN = 6, the update action is undone ie the contents of page P505 is changed to STUV

A CLR is written with undoNextLSN = prevLSN of 6 = 2.

The set ToUndo now becomes = { 2 }

At recLSN = 2, the update action is undone ie the contents of page P500 is changed to GABC

A CLR is written with undoNextLSN = prevLSN of 2 = NULL.